

Informe de Consulta y Guía Práctica: SolarBI Pascual

Estudiantes:

Isabella Aristizabal Diaz
Valentina Alejandra Pérez Cardona

Docente:

Ramiro Grisales Montoya

Asignatura:

Inteligencia de negocios (BI)

Institución:

Institución Universitaria Pascual Bravo

1. Parte A - Consulta Teórica Aplicada

Bloque 1 - Power BI y Grafana

1.1.1. ¿Qué es Grafana? Componentes y diferencias entre Grafana OSS y Cloud

Grafana es una plataforma para visualizar y analizar datos en tiempo real mediante piezas clave como fuentes de datos (ej. PostgreSQL), paneles con gráficos (panels), tableros organizados (dashboards), variables para filtrar información y alertas automáticas (alerting). Su versión OSS es gratuita e instalable en tus propios servidores, lo que te da control total pero exige que te encargues del mantenimiento y la infraestructura. Por otro lado, Grafana Cloud es la opción administrada en la nube por Grafana Labs; pagas una suscripción y ellos asumen la seguridad, las actualizaciones y la escalabilidad, ahorrándote tiempo de gestión.

(Fuente: Grafana Labs Documentation)

1.1.2. Tabla comparativa: Power BI vs. Grafana

Criterio	Power BI	Grafana
Propósito principal	Análisis de negocio, reporting corporativo e inteligencia empresarial.	Monitoreo operativo, telemetría e infraestructura en tiempo real.
Usuario típico	Analistas de datos, gerentes, ejecutivos y toma de decisiones de negocio.	Ingenieros DevOps, operadores de planta y personal técnico IoT.
Conexión a los datos	Importación programada (Import) o DirectQuery a bases relacionales.	Consulta directa y continua a bases temporales/relacionales.
Modelo semántico / Lenguaje	Modelo relacional (Estrella/Copo de nieve), lenguaje DAX y Power Query.	Sin modelo semántico integrado; lenguaje nativo de la fuente (SQL, PromQL).
Actualización y tiempo real	Actualizaciones programadas (máx. 8 a 48 veces/día según licencia).	Actualización en tiempo real (auto-refresh de milisegundos o segundos).
Alertas	Alertas en tableros basadas en umbrales simples (servicio Power BI).	Motor robusto de alertas multicanal (Slack, Email, PagerDuty).
Licenciamiento y costo	Licenciamiento por usuario/mes (Pro/Premium) o capacidad.	Grafana OSS es gratuito y de código abierto; Cloud según consumo.
Control de acceso	RLS (Row-Level Security) avanzado y permisos en workspace.	Gestión de roles (Admin, Editor, Viewer) y acceso por dashboard/carpeta.

(Fuente: Microsoft Learn / Grafana Documentation)

1.1.3. Tipos de Dashboards y Clasificación de KPIs de SolarBI

Los dashboards responden a tres necesidades: los operativos monitorean el día a día en tiempo real, los analíticos examinan tendencias históricas y los estratégicos miden metas del negocio.

En SolarBI, la Disponibilidad, Alarmas y el % de datos válidos son operativos y viven en Grafana por su agilidad en tiempo real. Por su parte, el Yield, Performance Ratio y la Energía generada son analíticos y se analizan en Power BI para evaluar la eficiencia histórica. Finalmente, el Ahorro financiero es estratégico y también está en Power BI para guiar las decisiones directivas y el retorno de inversión.

(Fuente: Few, S. "Information Dashboard Design")

1.1.4. Modos de almacenamiento en Power BI vs. Auto-refresh de Grafana

En Power BI, el modo Import guarda los datos en memoria para ofrecer máxima velocidad, aunque exige actualizaciones programadas; DirectQuery los consulta en tiempo real, pero sacrifica rapidez. Para SolarBI conviene un modelo híbrido o Import con actualización incremental, optimizando costos y rendimiento en los análisis avanzados de la capa Gold. Por el contrario, Grafana no almacena nada: su actualización automática lanza consultas directas a la base de datos a intervalos regulares (ej. cada 5 segundos), algo perfecto para monitorear la telemetría en tiempo real en las capas Bronze y Silver sin saturar memoria.

(Fuente: Microsoft Power BI Guidance / Grafana Docs)

1.1.5. Variables en Grafana vs. Segmentadores en Power BI

En Grafana, las variables modifican la consulta directamente en la fuente (como agregar un filtro \$dispositivo), lo que reduce el tráfico de red al traer solo los datos necesarios. En cambio, los segmentadores de Power BI filtran la información en memoria usando relaciones entre tablas y lenguaje DAX sobre el modelo ya cargado. Un claro ejemplo es la variable \$inversor en Grafana dentro de SolarBI: reestructura la consulta nativa para traer únicamente la serie temporal seleccionada, optimizando de forma drástica el ancho de banda y los recursos del servidor.

(Fuente: Grafana Docs / Power BI Documentation)

Bloque 2 · Gobernanza de Datos (Data Governance)

1.2.1. Definición de Data Governance, Data Management y Data Quality (DAMA-DMBOK)

Según DAMA-DMBOK, Data Governance es la dirección que fija políticas, roles y reglas sobre los datos. Data Management es la parte técnica que ejecuta esas normas en el día a día para conectar, guardar y mantener la información en todo su ciclo de vida. Por su parte, Data Quality se encarga de confirmar que los datos sean precisos, completos, oportunos y coherentes. En resumen: la gobernanza establece el “qué” y el “quién”, la gestión implementa el “cómo” y la calidad evalúa el “resultado”.

(Fuente: DAMA-DMBOK 2nd Edition)

1.2.2. Roles de Gobernanza vs. Roles del Equipo SolarBI

El Data Owner lidera el negocio y aprueba los casos de uso, rol que asume el Product Owner en SolarBI. El Data Steward vela por la calidad, definiciones y reglas del dato, responsabilidad dividida entre el BI Analyst y el Data Modeler al diseñar indicadores en Power BI. Por último, el Data Custodian se encarga del almacenamiento y la infraestructura segura, tarea que recae en el Data

Engineer para mantener los procesos ETL y PostgreSQL. En síntesis, la gobernanza fija los roles teóricos y el equipo de SolarBI los aterriza en la práctica.

(Fuente: DAMA-DMBOK 2nd Edition)

1.2.3. Contrato de Datos (Data Contract) en SolarBI

Un contrato de datos define el esquema, unidades de medida (kW, kWh), frecuencia de actualización (5 minutos), responsable del origen y niveles de calidad requeridos. Si la fuente cambia un campo sin avisar (por ejemplo, p_ac_kw por potencia_ac), los procesos ETL fallan de inmediato, rompiendo los reportes en Power BI y Grafana y provocando errores en cadena. Este acuerdo evita ese problema al garantizar cambios planificados por versiones e integración continua, manteniendo estables todos los sistemas.

(Fuente: Goedegebuure, "Data Contracts")

1.2.4. Gobernanza de Indicadores y Ficha de KPIs

Una Única Fuente de Verdad centraliza la información para que toda la empresa use las mismas cifras, mientras que un Catálogo de KPIs documenta la definición formal de cada métrica. Por ejemplo, el Yield (Energía/Potencia Nominal, en kWh/kWp) lo gestiona el BI Analyst a nivel diario, mientras que el % de Datos Válidos lo controla el Data Engineer cada 5 minutos. Documentar esto es clave, pues calcular un KPI de forma dual (en SQL y DAX) genera inconsistencias por diferencias en redondeos, filtros o zonas horarias.

Fichas de KPIs del Proyecto:

KPI	Fórmula	Unidad	Fuente	Responsable	Frecuencia
Yield	Energía (kWh) / Potencia Nominal (kWp)	kWh/kWp	dwh.fact_energia_dia	BI Analyst	Diaria
% Datos Válidos	$\left(\frac{\text{Filas Válidas}}{\text{Filas Totales}} \right) * 100$	%	silver.lectura_5min	Data Engineer	Cada 5 min

(Fuente: DAMA-DMBOK)

1.2.5. Ley 1581 de 2012 y Ética de Datos

Aunque los datos técnicos de SolarBI (potencia o irradiancia) no se consideran datos personales bajo la Ley 1581 de 2012, incluir al operador que atendió una alarma sí los convierte en información personal identificable. Ante esto, el equipo adopta una postura ética y legal enfocada en la minimización de datos, la anonimización y el uso de Row-Level Security (RLS) para resguardar la privacidad laboral de los trabajadores.

(Fuente: Ley 1581 de 2012 - Superintendencia de Industria y Comercio)

Bloque 3 · Automatización de Procesos ETL

1.3.1. ETL vs. ELT y Power Query

En ETL los datos se transforman antes de cargarlos al destino, mientras que en ELT se suben crudos para aprovechar la potencia de procesamiento de la base de datos. En SolarBI esto se refleja en sus capas:

- Bronze guarda la información cruda
- Silver aplica limpieza mediante ETL
- Gold genera agregaciones orientadas al negocio mediante ELT.

Power Query opera como ETL en entornos locales, pero evoluciona a un enfoque ELT al ejecutarse sobre Data Lakes en Power BI Service.

(Fuente: Kimball, R. "The Data Warehouse Toolkit")

1.3.2. Carga Completa vs. Incremental e Idempotencia

La carga completa reemplaza todo el histórico en cada ejecución, mientras que la incremental solo procesa los datos nuevos o modificados. Para que esta última sea segura debe ser constante, lo que significa que ejecutar el proceso varias veces dará exactamente el mismo resultado sin duplicar información. En la tabla fact_energia_dia esto se logra mediante una clave primaria compuesta y una operación UPSERT, garantizando que los registros existentes se actualicen en lugar de duplicarse.

(Fuente: Kleppmann, M. "Designing Data-Intensive Applications")

1.3.3. Comparativa de Herramientas de Automatización

Herramienta	¿Qué es?	¿Qué automatiza?	¿Cuándo conviene?
Cron / Programador de Tareas	Programador de tareas a nivel de sistema operativo.	Scripts locales (.py, .sh, .bat) en horarios fijos.	Flujos sencillos, un solo script y recursos locales mínimos.
Apache Airflow	Orquestador de flujos de trabajo programático (Python).	DAGs complejos, reintentos, dependencias y alertas.	Entornos empresariales, flujos ETL/ELT complejos y multiorigen.
Scheduled Refresh (Power BI)	Funcionalidad en la nube de Power BI Service.	Importación y actualización del modelo semántico.	Tableros finales orientados a usuarios de negocio.

(Fuente: Airflow Docs / Microsoft Learn)

1.3.4. Actualización Incremental en Power BI

La actualización incremental utiliza los parámetros RangeStart y RangeEnd para filtrar datos dinámicamente en el origen. Power BI organiza la información en particiones estáticas para el histórico y dinámicas para los periodos recientes, refrescando solo estas últimas. Resulta indispensable en modelos masivos con millones de filas, ya que reduce el tiempo de procesamiento, optimiza el uso de memoria en la puerta de enlace (gateway) y minimiza el impacto en la base de datos de origen.

(Fuente: Microsoft Power BI Documentation)

Bloque 4 · Internet de las Cosas (IoT)

1.4.1. Arquitectura IoT por Capas y Flujo de Datos

La arquitectura conecta dispositivos físicos con la capa visual pasando por cinco niveles: Inversor, Gateway/Edge, Broker MQTT, Almacenamiento y Aplicaciones. El flujo comienza cuando el inversor mide la potencia y la envía vía MQTT a la capa Bronze para guardarla cruda. Luego pasa a la capa Silver, donde se limpian inconsistencias y datos faltantes. Finalmente, la capa Gold consolida resúmenes diarios que alimentan los tableros de Power BI y Grafana para el análisis operativo y gerencial.

1.4.2. MQTT: Conceptos y Jerarquía de Topics

MQTT es un protocolo liviano basado en un patrón de Publicador/Suscriptor. Los dispositivos envían métricas a un Broker central organizadas bajo rutas de texto llamadas Topics, mientras que los clientes suscritos a esos temas las reciben al instante. Define tres niveles de garantía de entrega: QoS 0 (máximo una vez), QoS 1 (al menos una vez) y QoS 2 (exactamente una vez). Una estructura jerárquica clara, como pascualbravo/solar/medellin/inversor_01/telemetry, permite organizar las lecturas de forma lógica y escalable.

(Fuente: OASIS MQTT Standard)

1.4.3. Comparativa: Modbus vs. MQTT vs. OPC UA

Criterio	Modbus (RTU/TCP)	MQTT	OPC UA
Modelo	Maestro / Esclavo (Poll)	Publicador / Suscriptor	Cliente / Servidor
Uso típico	Comunicación local industrial / RS485.	Telemetría IoT en la nube y redes limitadas.	Interoperabilidad e integración SCADA.
Formato	Registros binarios / Hexadecimal.	JSON, Texto o Binario (Payload libre).	Estructurado / XML / Binario.
Seguridad	Nula por defecto.	TLS/SSL y autenticación básica.	Cifrado robusto, certificados x509.

En la planta: Se espera Modbus en los inversores/medidores locales, enviando los datos a un Edge Gateway que traduce la lectura a MQTT hacia la nube.

(Fuente: Industrial Internet Consortium)

1.4.4. Batch vs. Streaming y Granularidad

Procesar datos en lotes (Batch) reduce drásticamente el volumen almacenado, agiliza las consultas ejecutivas y reduce costos, aunque sacrifica la visibilidad de picos instantáneos y alertas en tiempo real. Por esta razón, las herramientas se complementan según el tipo de análisis: Grafana destaca en el procesamiento en flujo (Streaming a intervalos de 5 minutos) para el monitoreo operativo inmediato, mientras que Power BI aprovecha la granularidad agregada en lotes diarios dentro de la capa Gold para analizar tendencias históricas.

(Fuente: White, T. "Hadoop: The Definitive Guide")

Bloque 5 · Control de Versiones

1.5.1. Control de Versiones: Git, GitHub, Power BI (.pbip) y Grafana

Git es el sistema de control de versiones distribuido local, mientras que GitHub es la plataforma en la nube para alojar repositorios.

Conceptos: Repository (proyecto), Commit (guardado histórico), Branch (línea de desarrollo alternativa), Pull Request (solicitud para fusionar cambios).

Los archivos .pbix de Power BI son binarios zip comprimidos, imposibles de comparar en Git. El formato Power BI Project (.pbip) resuelve esto desglosando el informe en archivos de texto (JSON/TMDL), permitiendo ver diffs línea por línea. Los dashboards de Grafana se versionan exportando su estructura en formato nativo JSON.

(Fuente: *Pro Git Book / Microsoft Learn*)

Enlace a el repositorio de github donde se desarrollo toda la práctica:
<https://github.com/isaaris1/solarbi-aristizabal-perez>

Contributors:

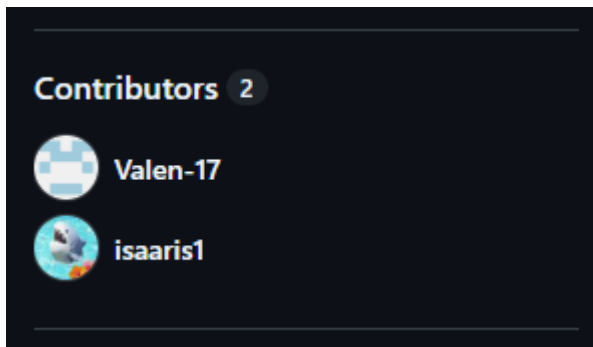
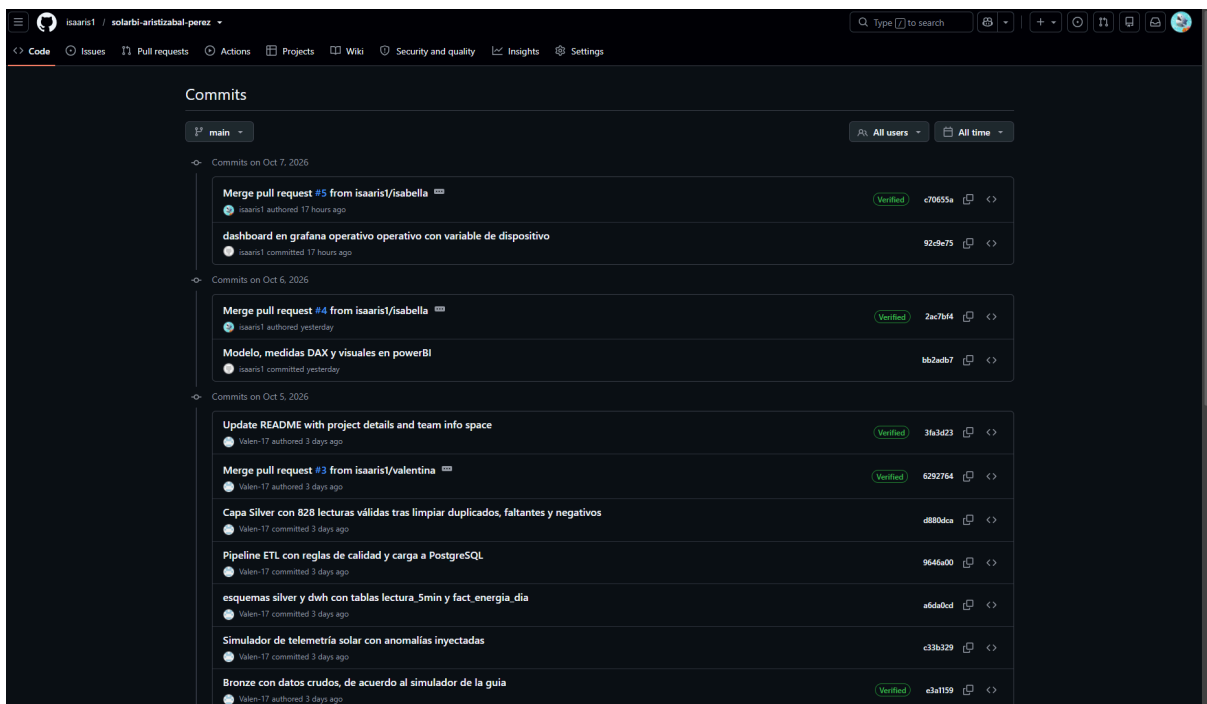


Imagen con historial de commits realizados antes de incluir este documento:



Se muestran commits de Valen-17 - Valentina Alejandra Perez Cardona e isaaris1 - Isabella Aristizabal Díaz.

2. Parte B · Guía de Implementación Práctica

Paso 1: Generador Bronze (etl/simulador.py)

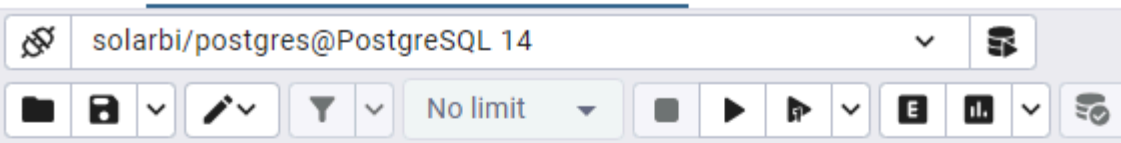
Guarda el código en el directorio `etl/simulador.py` para generar las lecturas sintéticas:

```
1 import csv, math, os, random
2 from datetime import datetime, timedelta
3
4 inicio = datetime(2026, 10, 5, 0, 0)
5 os.makedirs("data/bronze", exist_ok=True)
6
7 with open("data/bronze/telemetria.csv", "w", newline="", encoding="utf-8") as f:
8     w = csv.writer(f)
9     w.writerow(["ts", "dispositivo_id", "p_ac_kw", "irradiancia_wm2", "temp_modulo_c"])
10    for i in range(3 * 288): # 3 días, lecturas cada 5 min
11        ts = inicio + timedelta(minutes=5 * i)
12        sol = max(0.0, math.sin(math.pi * (ts.hour + ts.minute / 60 - 6) / 12))
13        irr = round(1000 * sol * random.uniform(0.7, 1.0), 1)
14        p_ac = round(5.0 * irr / 1000 * random.uniform(0.80, 0.90), 3)
15        fila = [ts.isoformat(sep=" "), 1, p_ac, irr, round(22 + 30 * sol, 1)]
16        r = random.random()
17        if r < 0.02:
18            fila[2] = -1 # Anomalía: Potencia negativa
19        elif r < 0.04:
20            fila[3] = "" # Anomalía: Dato faltante
21        w.writerow(fila)
22        if r > 0.98:
23            w.writerow(fila) # Anomalía: Fila duplicada
24
25 print("Archivo Bronze generado exitosamente.")
26
```

Paso 2 y 3: Configuración de Base de Datos y Pipeline ETL Idempotente

1. Estructura SQL en PostgreSQL (`sql/init.sql`):

Welcome  solarbi/postgres@PostgreSQL 14* X



Query Query History

```
1 CREATE SCHEMA IF NOT EXISTS silver;
2 CREATE SCHEMA IF NOT EXISTS dwh;
3
4 CREATE TABLE IF NOT EXISTS silver.lectura_5min (
5     ts TIMESTAMP PRIMARY KEY,
6     dispositivo_id INT,
7     p_ac_kw NUMERIC,
8     irradiancia_wm2 NUMERIC,
9     temp_modulo_c NUMERIC
10 );
11
12 CREATE TABLE IF NOT EXISTS dwh.fact_energia_dia (
13     fecha DATE,
14     dispositivo_key INT,
15     energia_kwh NUMERIC,
16     pct_datos_validos NUMERIC,
17     PRIMARY KEY (fecha, dispositivo_key)
18 );
19
```

2. Punto de Entrada Unificado (etl/run_etl.py):

```

1 import os
2 import pandas as pd
3 from sqlalchemy import create_engine, text
4 from dotenv import load_dotenv
5
6 load_dotenv()
7
8 DB_USER = os.getenv("DB_USER", "postgres")
9 DB_PASS = os.getenv("DB_PASS", "postgres")
10 DB_HOST = os.getenv("DB_HOST", "localhost")
11 DB_PORT = os.getenv("DB_PORT", "5432")
12 DB_NAME = os.getenv("DB_NAME", "solarbi")
13
14 engine = create_engine(f"postgresql://{DB_USER}:{DB_PASS}@{DB_HOST}:{DB_PORT}/{DB_NAME}")
15
16 def run_pipeline():
17     print("--- INICIANDO PIPELINE ETL ---")
18     bronze_path = "data/bronze/telemetry.csv"
19     if not os.path.exists(bronze_path):
20         raise FileNotFoundError("Ejecuta primero simulador.py")
21
22     df = pd.read_csv(bronze_path)
23     total_filas = len(df)
24
25     # Reglas de Calidad
26     duplicados = df.duplicated().sum()
27     df_clean = df.drop_duplicates()
28
29     faltantes = df_clean['irradiancia_wm2'].isna().sum()
30     df_clean = df_clean.dropna(subset=['irradiancia_wm2'])
31
32     invalidos_p = (df_clean['p_ac_kw'] < 0).sum()
33     df_clean = df_clean[df_clean['p_ac_kw'] >= 0]
34
35     filas_validas = len(df_clean)
36     pct_validos = round((filas_validas / total_filas) * 100, 2)
37
38     print(f"Filas Leídas: {total_filas} | Válidas: {filas_validas} ({pct_validos}%")
39
40     # Guardar en Silver Local
41     os.makedirs("data/silver", exist_ok=True)
42     df_clean.to_csv("data/silver/telemetry_limpia.csv", index=False)
43
44     # Carga Idempotente a PostgreSQL - Silver (Lectura 5min)
45     with engine.begin() as conn:
46         for _, row in df_clean.iterrows():
47             stmt = text("""
48                 INSERT INTO silver.lectura_5min (ts, dispositivo_id, p_ac_kw, irradiancia_wm2, temp_modulo_c)
49                 VALUES (:ts, :disp, :pac, :irr, :temp)
50                 ON CONFLICT (ts) DO UPDATE SET
51                     p_ac_kw = EXCLUDED.p_ac_kw,
52                     irradiancia_wm2 = EXCLUDED.irradiancia_wm2,
53                     temp_modulo_c = EXCLUDED.temp_modulo_c;
54             """)
55             conn.execute(stmt, {
56                 "ts": row['ts'], "disp": int(row['dispositivo_id']),

```

```

56         "ts": row['ts'], "disp": int(row['dispositivo_id']),
57         "pac": float(row['p_ac_kw']), "irr": float(row['irradiancia_wm2']),
58         "temp": float(row['temp_modulo_c'])
59     })
60
61     # Transformación a Gold (Cálculo diario)
62     df_clean['fecha'] = pd.to_datetime(df_clean['ts']).dt.date
63     df_clean['energia_intervale_kwh'] = df_clean['p_ac_kw'] * (5.0 / 60.0)
64
65     gold_df = df_clean.groupby(['fecha', 'dispositivo_id']).agg(
66         energia_kwh=('energia_intervale_kwh', 'sum')
67     ).reset_index()
68     gold_df['pct_datos_validos'] = pct_validos
69
70     # Carga Idempotente a PostgreSQL - Gold (fact_energia_dia)
71     with engine.begin() as conn:
72         for _, row in gold_df.iterrows():
73             stmt = text("""
74                 INSERT INTO dwh.fact_energia_dia (fecha, dispositivo_key, energia_kwh, pct_datos_validos)
75                 VALUES (:fecha, :disp, :energia, :pct)
76                 ON CONFLICT (fecha, dispositivo_key) DO UPDATE SET
77                     energia_kwh = EXCLUDED.energia_kwh,
78                     pct_datos_validos = EXCLUDED.pct_datos_validos;
79             """)
80             conn.execute(stmt, {
81                 "fecha": row['fecha'], "disp": int(row['dispositivo_id']),
82                 "energia": float(row['energia_kwh']), "pct": float(row['pct_datos_validos'])
83             })
84
85     print("--- PIPELINE COMPLETADO EXITOSAMENTE ---")
86
87 if __name__ == "__main__":
88     run_pipeline()
89

```

Configuración de Cron:

```

PS> schtasks /Query /TN SolarBI_ETL /V /FO LIST

Nombre de tarea:          \SolarBI_ETL
Hora próxima ejecución:  9/10/2026 12:00:00 a. m.
Último tiempo de ejecución: 8/10/2026 8:32:30 a. m.
Último resultado:        0
Tarea que se ejecutará:  cmd /c cd /d C:\Users\isabe\Proyectos-universidad\solarbi-aristizabal-perez\solarbi-aristizabal-perez && python etl\run_etl.py >> etl\etl.log 2>&1
Estado de tarea programada: Habilitado
Tipo de programación:    Diariamente
Hora de inicio:          12:00:00 a. m.
Fecha de inicio:         8/10/2026
Días:                    Cada 1 día(s)

PS> Get-Content etl\etl.log
--- INICIANDO PIPELINE ETL ---
Filas Leídas: 889 | Válidas: 828 (93.24%)
--- PIPELINE COMPLETADO EXITOSAMENTE ---

PS C:\Users\isabe\Proyectos-universidad\solarbi-aristizabal-perez\solarbi-aristizabal-perez>

```

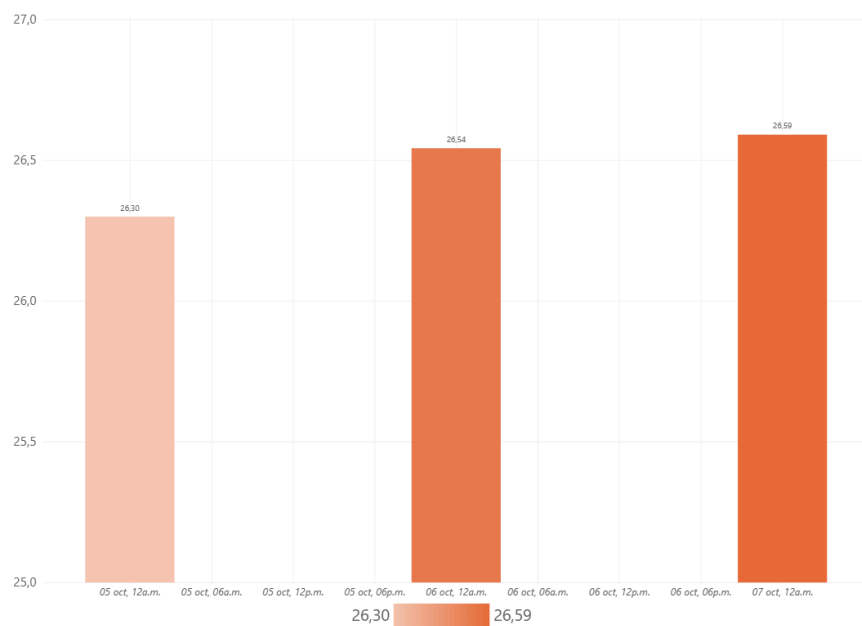
Paso 4: Construcción en Power BI

- Conectar Power BI Desktop a PostgreSQL (localhost:5432/solarbi) en modo Import.
- Crear Parámetro de Tarifa: TarifaCOP = 800 (COP/kWh, tarifa comercial promedio CREG en Colombia).
- Medidas DAX:

✗ ✓	1 Ahorro_COP = [Total_Energia_kWh] * [TarifaCOP]
✗ ✓	1 Total_Energia_kWh = SUM('dwh fact_energia_dia'[energia_kwh])
✗ ✓	1 Yield_kWh_kwp = DIVIDE([Total_Energia_kWh], 5.0, 0)

- Visuales: Tarjeta con Yield, Gráfico de columnas con energía diaria y Tarjeta con Ahorro en COP.

Energía generada por día (kWh)
Total_Energia_kWh por fecha



Yield_kWh_kWp

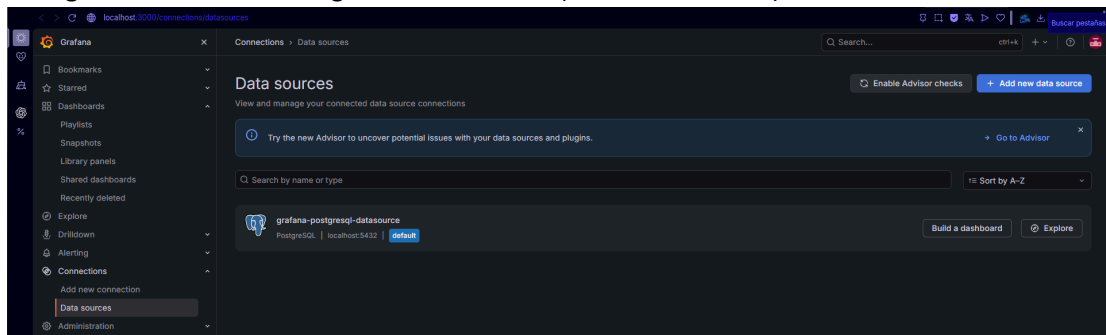
15,89

Ahorro_COP

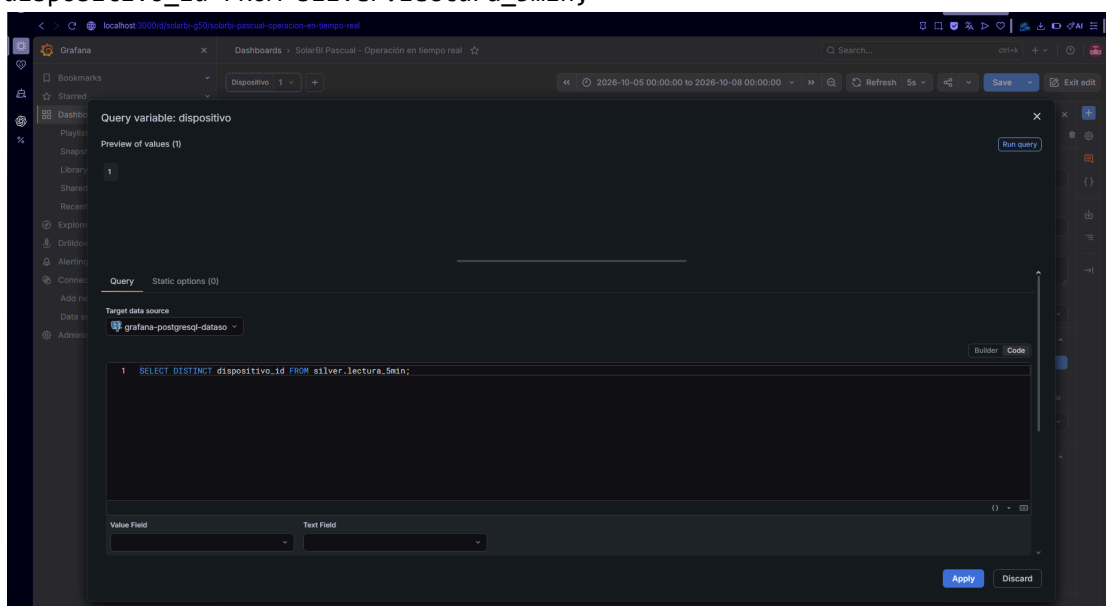
\$ 63,546 mil

Paso 5: Construcción en Grafana

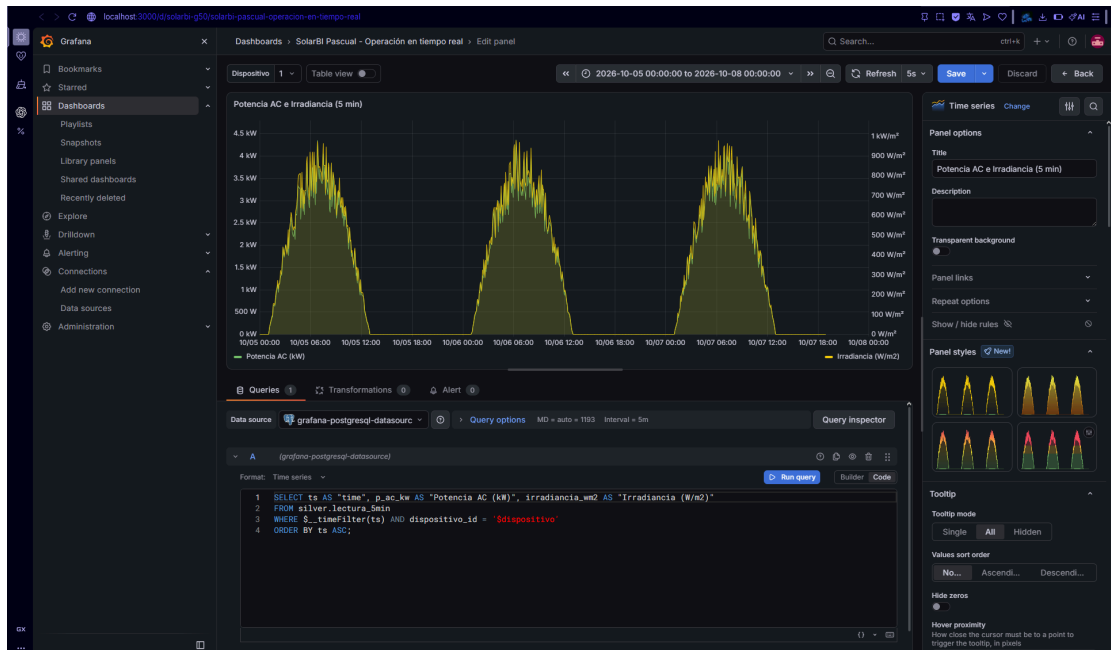
- Configurar data source PostgreSQL en Grafana (localhost:5432).



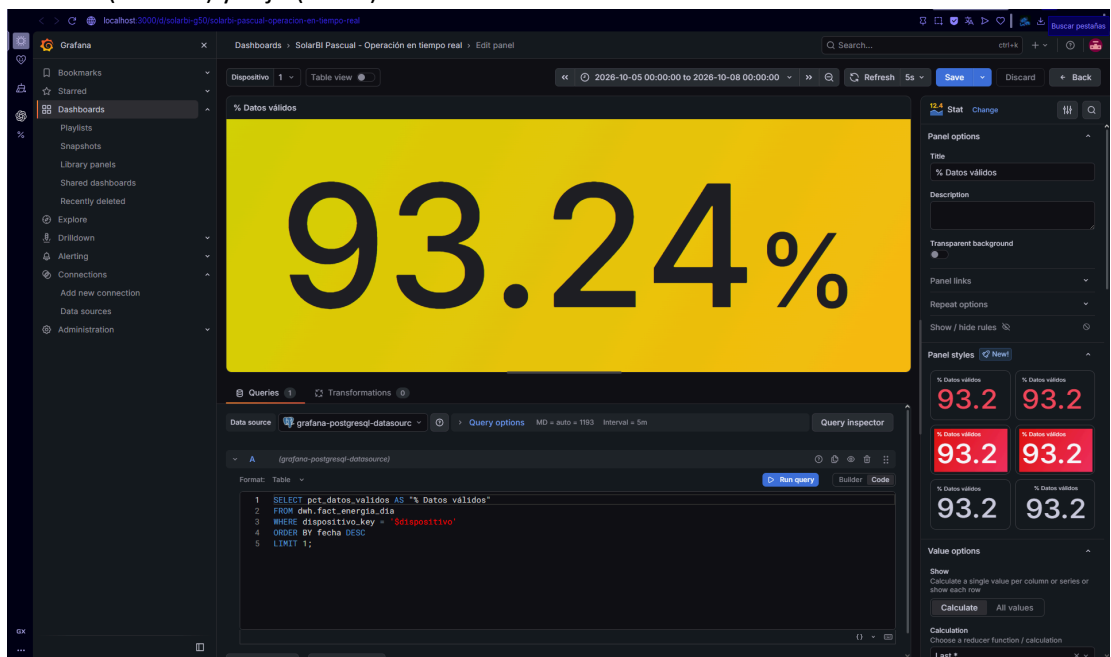
- Crear variable de dashboard \$dispositivo con la consulta: `SELECT DISTINCT dispositivo_id FROM silver.lectura_5min;`



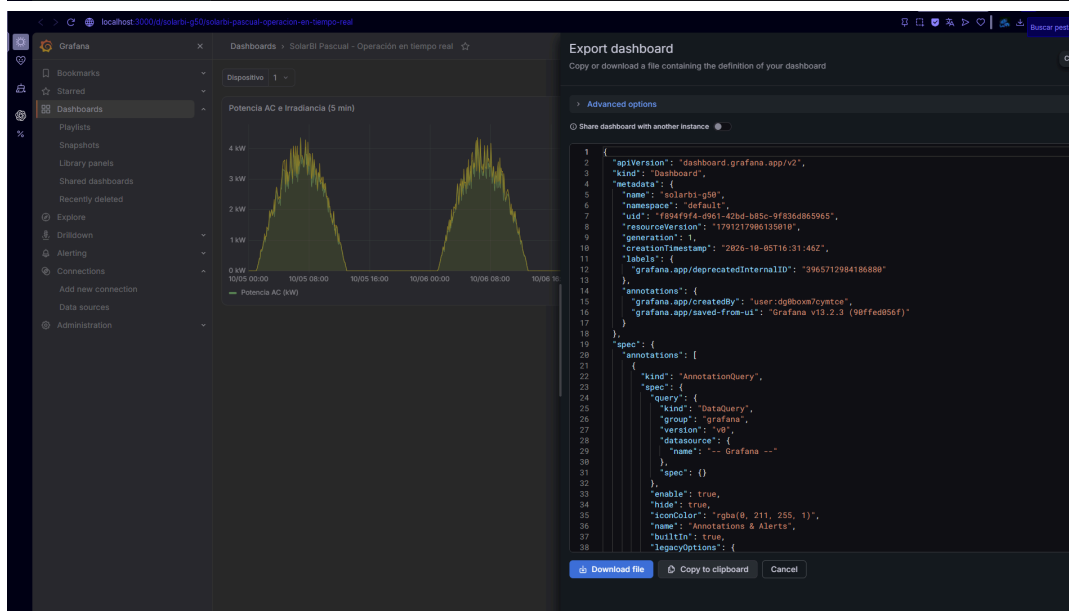
- Panel 1 (Series de tiempo):



- Panel 2 (Stat/Gauge): Consulta de pct_datos_validos con umbrales en color verde (>95%), amarillo (80-95%) y rojo (<80%).



- Exportar tablero como grafana/dashboard.json.



Paso 6: Reflexión Comparativa

Grafana resolvió con mayor eficacia el análisis temporal continuo de telemetría a 5 minutos gracias al soporte de macros nativas de tiempo. Power BI destacó en el modelado semántico, cálculo de KPIs consolidados en DAX y representación gráfica orientada al negocio (ahorro en COP). La mayor dificultad radicó en dinamizar las consultas SQL en Grafana mediante variables, y en estructurar parámetros en Power BI. Para un operador de planta se recomienda Grafana por el monitoreo continuo, y para un directivo se recomienda Power BI por el análisis gerencial y estratégico.

"One governed dataset, two views: Grafana for real-time operations and Power BI for business decisions."

3. Referencias

Apache Software Foundation. (s. f.). *Apache Airflow documentation*. Apache Airflow. <https://airflow.apache.org/docs/>

Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. <https://git-scm.com/book/en/v2>

DAMA International. (2017). *DAMA-DMBOK: Data management body of knowledge* (2nd ed.). Technics Publications.

Few, S. (2013). *Information dashboard design: Displaying data for at-a-glance monitoring* (2nd ed.). Analytics Press.

Goedegebuure, A. (2023). *Data contracts*. Anndata. <https://anndata.nl/data-contracts/>

Grafana Labs. (s. f.). *Grafana documentation*. Grafana Labs. <https://grafana.com/docs/grafana/latest/>

Industry IoT Consortium. (s. f.). *Industrial Internet Consortium resource hub*. Industry IoT Consortium. <https://www.iiconsortium.org/>

Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3rd ed.). John Wiley & Sons.

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.

Microsoft. (s. f.). *Power BI documentation and guidance*. Microsoft Learn. <https://learn.microsoft.com/es-es/power-bi/>

OASIS. (2019). *MQTT version 5.0*. OASIS Standard. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>

Superintendencia de Industria y Comercio. (2012). *Ley Estatutaria 1581 de 2012: Por la cual se dictan disposiciones generales para la protección de datos personales*. Congreso de la República de Colombia. <https://www.sic.gov.co/sobre-la-protección-de-datos-personales>

White, T. (2015). *Hadoop: The definitive guide* (4th ed.). O'Reilly Media.